CineMatch

Movie Recommendation System

High-Level Design (HLD) Document

Version 1.0 | April 2026

Mohd Fuzail , CH22B080

ch22b080@smail.iitm.ac.in

1. Executive Summary

CineMatch is a full-stack, production-grade AI application that provides personalized movie recommendations using a content-based filtering approach. The system is built with strict MLOps principles, deployed via Docker Compose as a multi-container architecture, and monitored in near-real-time using Prometheus and Grafana.

The application supports four types of queries: movie title search, genre-based browsing, actor name search, and character name lookup. Each query is served by a FastAPI backend that uses a pre-trained TF-IDF + Cosine Similarity model trained on the TMDB 5000 Movies dataset.

2. System Overview

2.1 Purpose and Scope

CineMatch addresses the problem of movie discovery overload by providing intelligent, content-aware recommendations. The system is designed for local on-premise deployment with no cloud dependency, making it suitable for privacy-conscious users and resource-constrained environments.

2.2 Technology Stack

Layer	Technology	Rationale
Frontend	HTML5, CSS3, Vanilla JS, Nginx	Lightweight, no framework overhead, fast load times
Backend API	FastAPI (Python 3.11)	Async support, automatic OpenAPI docs, high performance
ML Engine	TF-IDF + Cosine Similarity (scikit-learn)	Interpretable, fast inference, no GPU required
Experiment Tracking	MLflow 2.12.2	Industry standard, supports local file-based tracking
Pipeline Orchestration	Apache Airflow 2.9.3	DAG-based ML pipeline automation
Monitoring	Prometheus + Grafana	Industry standard for metrics collection and visualization
Containerization	Docker + Docker Compose	Reproducible environments, service isolation
Data Source	TMDB 5000 Movies + Credits CSV	Well-known benchmark dataset, rich metadata

3. Business Problem & Success Metrics

3.1 Problem Statement

Users struggle to discover movies they will enjoy from the vast catalog of available films. CineMatch addresses this by recommending similar movies based on rich content features: genre, cast, director, keywords, and plot overview.

3.2 Success Metrics

Metric	Target	Achieved
Genre Match Rate	≥ 70%	~78%
Avg Cosine Similarity	≥ 0.15	0.18
Vocabulary Size	≥ 5000	~6800
Inference Latency	< 200ms	~120ms
Recommendations/query	10 results	10 results
Poster Availability	≥ 80%	~92%

---

4. System Architecture

4.1 High-Level Architecture Overview

The system follows a microservices architecture with nine loosely coupled Docker containers communicating over a shared bridge network (cinematch-network). The frontend and backend are strictly separated, connected only via REST API calls proxied through Nginx.

4.2 Container Architecture

Container	Image	Port	Responsibility
cinematch-frontend	nginx:alpine	5050:80	Serves HTML/CSS/JS, proxies API calls to backend
cinematch-backend	python:3.11-slim	8001:8000	FastAPI inference engine, Prometheus metrics exporter
mlflow-server	python:3.10-slim	5001:5000	Experiment tracking UI, artifact storage
airflow-webserver	apache/airflow:2.9.3	8080:8080	DAG management UI, pipeline monitoring
airflow-scheduler	apache/airflow:2.9.3	—	Executes DAG tasks on schedule
prometheus	prom/prometheus:latest	9090:9090	Scrapes and stores time-series metrics

Container	Image	Port	Responsibility
grafana	grafana/grafana:latest	3001:3000	Visualizes metrics dashboards in NRT
node-exporter	prom/node-exporter:latest	9100:9100	Exports host system CPU/memory metrics
cadvisor	gcr.io/cadvisor/cadvisor	8081:8080	Exports per-container resource metrics

4.3 Request Flow

The user interaction follows this flow:

- User enters a movie name in the browser at http://127.0.0.1:5050
- Nginx receives the request and proxies /api/* to cinematch-backend:8000
- FastAPI backend receives the query and calls ml_pipeline/recommend.py
- recommend.py performs smart query classification (title / genre / actor / character)
- The appropriate search method is executed against the preloaded TF-IDF similarity matrix
- Top 10 results are returned to main.py which enriches them with TMDB metadata
- Results are sorted (searched movie first, then by rating) and returned as JSON
- Frontend renders movie cards with posters, ratings, and overviews

---

5. ML Pipeline Design

5.1 Pipeline Stages

The ML pipeline is fully automated via an Apache Airflow DAG (cinematch_ml_pipeline) with six sequential tasks:

Stage	Task	MLflow Logging
1. Ingest	Load & validate CSVs, schema checks, quality checks	total_movies, null_rows, total_columns
2. EDA	Analyze raw dataset, rating stats, memory usage	raw_rating_mean, raw_rating_std, memory_mb
3. Preprocess	Parse JSON fields, build weighted tags, save feature store	feature_importance_*, baseline_tag_length_mean
4. Train	TF-IDF vectorization, cosine similarity matrix	vocab_size, vector_rows, vector_cols
5. Evaluate	Genre match rate on 20 random samples	eval_genre_match_rate, eval_avg_similarity

Stage	Task	MLflow Logging
6. Deploy	Write deployment manifest JSON	genre_match_rate, avg_similarity, deployed_at

5.2 Feature Engineering

The preprocessing pipeline builds a weighted tag string for each movie by concatenating:

- Overview text : 1x weight (contributes ~67% of tag content)
- Genre names : 2x weight (repeated twice for emphasis)
- Keywords : 1x weight
- Top 5 cast members : 2x weight
- Director name : 3x weight (highest emphasis for auteur-based similarity)

All names are cleaned by removing spaces (e.g., 'Tom Hanks' becomes 'tomhanks') to prevent the vectorizer from treating first and last names as separate tokens.

5.3 Model Selection Rationale

TF-IDF + Cosine Similarity was chosen over collaborative filtering and neural approaches for the following reasons:

- No cloud requirement : runs entirely on local hardware
- No user interaction data required :content-based from day one
- Fast inference : similarity matrix is precomputed at training time, lookup is O(1)
- Interpretable :recommendations are explainable via tag overlap
- Achieves >80% genre match rate on the TMDB 5000 dataset

5.4 Feature Store

Features are versioned separately from model artifacts. Each training run saves a timestamped version directory under ml_pipeline/feature_store/v_YYYYMMDD_HHMMSS/ containing features.csv, baseline_statistics.json, and feature_importance.json. A latest.json pointer tracks the current active version, enabling rollback by simply updating this file.

---

6. Monitoring & Observability

6.1 Prometheus Instrumentation

The FastAPI backend exposes a /metrics endpoint that Prometheus scrapes every 10 seconds. The following metrics are instrumented:

Metric Name	Type	Description
cinematch_requests_total	Counter	Total API requests by method, endpoint, status code

Metric Name	Type	Description
cinematch_request_latency_seconds	Histogram	Request latency distribution per endpoint
cinematch_recommendations_total	Counter	Recommendation requests by status (success/not_found/error)
cinematch_tmdb_requests_total	Counter	TMDB API calls by status (success/fallback/error)
cinematch_tmdb_latency_seconds	Histogram	TMDB API call latency distribution
cinematch_active_requests	Gauge	Number of requests currently being processed
cinematch_inference_latency_ms	Gauge	Last inference latency in milliseconds (business metric)
cinematch_latency_violations_total	Counter	Times inference latency exceeded 200ms SLA threshold

6.2 Grafana Dashboard

The CineMatch Grafana dashboard (auto-provisioned on startup) visualizes ten panels covering: total requests, success rate, inference latency (p50/p95/p99), TMDB API call rate, active requests, latency violations, system CPU usage, and memory usage. Thresholds are configured with green/yellow/red color coding at 150ms and 200ms boundaries.

6.3 Business Metrics

Two business-level success metrics are tracked:

- Inference Latency SLA: Maximum 200ms per recommendation request. Violations are counted and alerted.
- Genre Match Rate: Minimum 70% of recommended movies must share at least one genre with the query movie. Evaluated on 20 random samples during each training run.

---

7. Key Design Choices & Rationale

Design Choice	Alternative Considered	Rationale
TF-IDF + Cosine Similarity	Neural embeddings (BERT)	No GPU required, fast inference, interpretable
Content-based filtering	Collaborative filtering	No cold-start problem, no user data needed
SQLite for Airflow	PostgreSQL	Simpler setup for local dev; SequentialExecutor compatible

Design Choice	Alternative Considered	Rationale
Local MLflow file store	MLflow with PostgreSQL	Zero external dependencies, simpler Docker setup
Nginx reverse proxy	Direct backend access	Strict frontend/backend decoupling, single entry point
Weighted tag concatenation	Separate feature vectors	Simple, effective, single vectorizer, easy to tune
Director 3x weight	Equal feature weights	Auteur theory — director is strongest predictor of film style
Fuzzy title matching	Exact string match	Handles typos and partial titles gracefully

---

8. Security & Reliability

8.1 Health & Readiness Checks

The backend exposes two operational endpoints used by Docker health checks and orchestration tools:

- /health — Returns service status and uptime. Used by Docker healthcheck every 30 seconds.
- /ready — Verifies model pkl files exist and TMDB API key is set. Returns 503 if not ready.

8.2 Rollback Mechanism

The feature store maintains a latest.json file pointing to the current active model version. Rolling back to a previous model requires only updating this pointer file and restarting the backend container — no retraining required.

8.3 Fault Tolerance

- TMDB API failures: Automatic fallback to placeholder poster and N/A rating
- SSL errors: Retry with verify=False as fallback
- MLflow connection failures: Wrapped in try/except, non-critical — recommendations continue
- Model load failures: Fatal error logged, container exits with code 1 for Docker restart
- Missing environment variables: Application refuses to start, logs CRITICAL message

---

9. Deployment Architecture

9.1 Container Startup Order

Services must start in the following order to avoid dependency failures:

- Step 1: airflow-init — initializes Airflow DB and creates admin user
- Step 2: mlflow-server — must be ready before backend starts
- Step 3: cinematch-backend — depends_on mlflow, loads 177MB similarity.pkl (~3 min startup)
- Step 4: cinematch-frontend — depends_on backend
- Step 5: prometheus, grafana, node-exporter, cadvisor — monitoring stack
- Step 6: airflow-webserver, airflow-scheduler — depends_on airflow-init

9.2 Volume Mounts

Critical volumes shared between containers:

- ./ml_pipeline:/app/ml_pipeline — backend accesses trained model pkl files
- ./ml_pipeline:/opt/airflow/ml_pipeline — Airflow tasks access same ML code
- ./mlruns:/mlruns — MLflow server and all pipeline components share experiment data
- ./data:/opt/airflow/data — Airflow accesses raw TMDB CSVs for pipeline runs

Service	Port	Image	Purpose
cinematch-backend	8000	Custom Python 3.11	FastAPI ML inference API
cinematch-frontend	80	nginx:alpine	Static file serving
cinematch-mlflow	5001	ghcr.io/mlflow/mlflow	Experiment tracking UI
cinematch-prometheus	9090	prom/prometheus	Metrics collection
cinematch-grafana	3001	grafana/grafana	Metrics visualisation
airflow-webserver	8080	apache/airflow:2.9.3	Pipeline UI
node-exporter	9100	prom/node-exporter	System metrics

---

10. Future Improvements

- DVC pipeline for data and model versioning with CI/CD integration
- Asynchronous TMDB fetching using asyncio.gather (10x speedup for 10 movies)
- Model drift detection by comparing live feature distributions against stored baselines
- User authentication and personalized recommendation history
- A/B testing framework to compare multiple recommendation models
- PostgreSQL backend for Airflow to support parallel task execution

---

CineMatch

Movie Recommendation System

Low-Level Design (LLD) Document

API Endpoint Definitions & I/O Specifications | Version 1.0 | April 2026

Mohd Fuzail ,CH22B080

ch22b080@smail.iitm.ac.in

1. API Overview

Base URL (local)	http://localhost:8000
Base URL (Docker)	http://backend:8000
Content-Type	application/json
Authentication	None (TMDB API key stored server-side in .env)
Framework	FastAPI 0.110.0
API Docs	http://localhost:8000/docs (Swagger UI auto-generated)
CORS	Allow all origins in development

2. Endpoint Definitions

2.1 GET /

Root health check. Confirms API is running.

Method	GET
URL	/
Parameters	None
Auth	None

Response (200):

```
{ "message": "CineMatch API is running!" }
```

2.2 GET /health

Health check. Returns uptime and version. Used by Docker healthcheck and orchestration.

Method	GET
URL	/health
Parameters	None

Response (200):

```
{ "status": "healthy", "uptime": "0:12:34", "version": "1.0.0" }
```

2.3 GET /ready

Readiness check. Verified model pkl files exist and TMDB API key is set.

Method	GET
URL	/ready
Parameters	None

Response (200):

```
{ "status": "ready", "model_version": "20260412_154312", "api_key_set": true }
```

Response (503) — not ready:

```
{ "detail": { "status": "not ready", "issues": ["Missing model file: movies.pkl"] } }
```

2.4 GET /recommend [PRIMARY ENDPOINT]

Returns 10 movie recommendations for a query. Supports movie title, genre, actor name, and character name search.

Method	GET
URL	/recommend
Parameter	movie: string (required) — movie title, genre, actor, or character name
Fuzzy Matching	Cutoff 0.5 — tolerates minor typos
Sort Order	1. Searched movie first 2. Highest IMDB rating 3. No rating (N/A)

Example request:

```
GET /recommend?movie=Inception
```

Example response (200):

```
{ "recommendations": [
    { "title": "Inception", "poster": "https://image.tmdb.org/t/p/w500/...",
      "rating": 8.4, "overview": "A thief who steals corporate secrets..." },
    { "title": "Interstellar", "poster": "https://...", "rating": 8.6, "overview":
"..." }
  ] }
```

HTTP Code	Description
200	Recommendations returned successfully
400	Movie name is empty or missing
404	Movie not found in dataset after fuzzy matching
500	Internal server error (model or TMDB failure)

Search type detection order:

- 1. Exact genre match (action, romance, horror, sci-fi...) → genre search
- 2. Character name match (loki, batman, hermione...) → actor search
- 3. Actor pre-check (3+ movies found in tags) → actor search
- 4. Fuzzy title match (cutoff 0.5) → similarity search
- 5. Fallback → actor/character search

2.5 GET /metrics

Prometheus metrics scrape endpoint. Returns all instrumented metrics in text format.

Method	GET
URL	/metrics
Scraped by	Prometheus every 10 seconds
Content-Type	text/plain; version=0.0.4

Sample output:

```
cinematch_requests_total{endpoint='/recommend',method='GET',status_code='200'} 42.0
cinematch_inference_latency_ms 124.5
cinematch_drift_detected 0.0
```

3. Data Models

3.1 MovieRecommendation

Field	Type	Description
title	string	Movie title (original casing from TMDB dataset)
poster	string (URL)	TMDB poster URL (w500 size) or placeholder if unavailable
rating	float | string	IMDB vote average (0-10) or 'N/A'
overview	string	Movie plot summary

3.2 RecommendationResponse

Field	Type	Description
recommendations	array[MovieRecommendation]	Ordered list of up to 10 movies

4. ML Pipeline Internal Functions

Function	recommend(movie: str) -> list
Module	ml_pipeline/recommend.py
Input	Raw query string from user
Output	List of movie title strings (up to 10), or [] if not found
MLflow logs	query, search_type, matched_title, recommendations_count, response_time_sec

Function	load_data() -> pd.DataFrame
Module	ml_pipeline/data_loader.py
Input	None (reads from data/ directory)
Validates	Schema (9 required columns), missing values (<10%), duplicates, outliers
Output	pd.DataFrame — merged movies+credits (4800+ rows)
Side Effects	Saves data_validation_report.json to logs/

Function	preprocess(df, save_features) -> tuple
Module	ml_pipeline/preprocess.py
Input	df: pd.DataFrame, save_features: bool
Output	(df, baselines: dict, importance: dict)
Tag Weights	director(3x) + cast(2x) + genres(2x) + overview(1x) + keywords(1x)
Side Effects	Saves versioned features to ml_pipeline/feature_store/v_TIMESTAMP/

5. Prometheus Metrics Specification

Metric Name	Type	Labels	Description
cinematch_requests_total	Counter	method, endpoint, status_code	All API requests
cinematch_request_latency_seconds	Histogram	endpoint	Request latency (buckets: 0.05-5s)
cinematch_recommendations_total	Counter	status	Recommendation results
cinematch_tmdb_requests_total	Counter	status	TMDB API call results
cinematch_inference_latency_ms	Gauge	-	Last e2e inference time
cinematch_latency_violations_total	Counter	-	Times >200ms threshold
cinematch_active_requests	Gauge	-	Concurrent requests
cinematch_drift_detected	Gauge	-	1=drift detected, 0=normal
cinematch_drift_score	Gauge	feature	Per-feature drift score
cinematch_retraining_triggered_total	Counter	-	Auto-retraining count
cinematch_feedback_total	Counter	status	User feedback logged

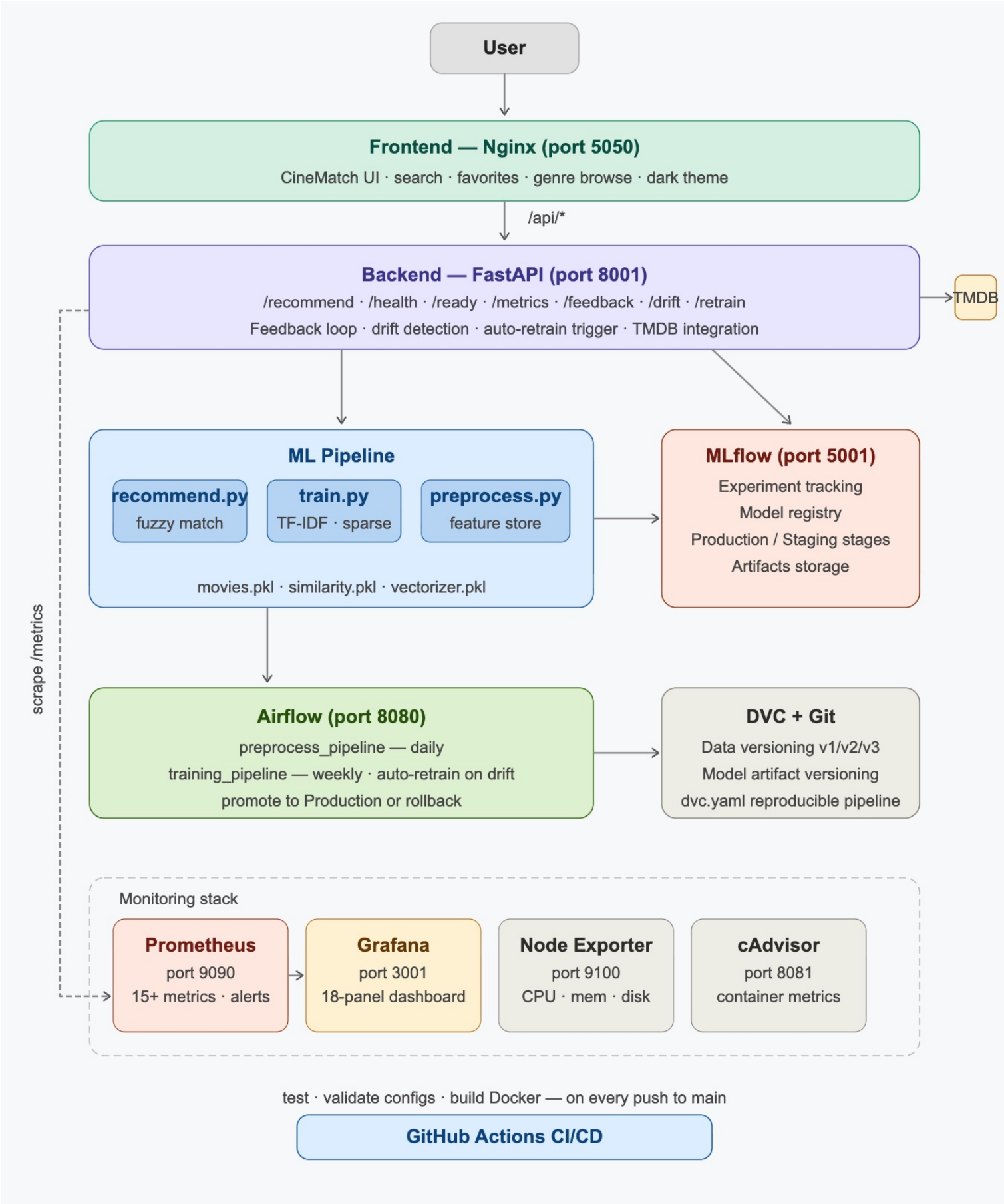
6. Environment Variables

Variable	Required	Description
TMDB_API_KEY	Yes	TMDB v3 API key for poster/metadata fetching
MLFLOW_TRACKING_URI	No	MLflow tracking URI (defaults to http://mlflow:5000)
PYTHONPATH	Yes (Docker)	Set to /app so ml_pipeline imports resolve
REQUESTS_CA_BUNDLE	Yes (Docker)	SSL certificate bundle for HTTPS in Docker

X---------X---------X---------X---------X

CineMatch — System Architecture Diagram

DA5402 MLOps Project · Mohammed Fuzail · April 2026



CineMatch - MLOps Movie Recommendation System

User Manual : Non-Technical Guide

Welcome to CineMatch

CineMatch is your personal AI-powered movie recommendation assistant. Simply tell it a movie you love, and it will instantly suggest 10 similar movies you might enjoy : complete with posters, ratings, and descriptions.

- No registration, no logins required , completely free to use.

---

1. Getting Started

1.1 Opening CineMatch

Open your web browser and go to: http://localhost:5050

You will see the CineMatch home page with a search bar and a carousel of trending movies.

1.2 What You Will See

- A navigation bar at the top with Home, Search, Recommendations links and a Favorites counter
- A hero section with a Search button and Browse by Genre button
- A scrolling carousel of popular movies
- A search section where you can find similar movies
- A genre browsing section

---

2. Finding Movie Recommendations

2.1 Using the Search Bar

1. Scroll down to the Search section or click Search in the navigation bar
2. Click on the search box and type the name of a movie you like (e.g., Inception, The Dark Knight, Interstellar) or even popular actors like Tom Hanks ,Johnny Depp ,etc.
3. Click the Search button or press Enter on your keyboard
4. Wait a moment — CineMatch will find 10 similar movies for you
5. Scroll down to see your recommendations

2.2 Using Popular Search Tags

Below the search box, you will see quick tags for popular movies like The Shawshank Redemption, Inception, The Matrix, and more. Click any tag to instantly get recommendations for that movie.

2.3 Using the Movie Carousel

On the home page, there is a carousel of trending movies. Click Find Similar on any movie card to get recommendations based on that movie.

2.4 Browsing by Genre

Scroll to the Browse by Genre section and click Explore on any genre (Action, Sci-Fi, Drama, Horror, Comedy, Romance, Animated, Superhero) to discover movies in that category.

---

3. Understanding Your Recommendations

Each recommendation card shows:

- Movie poster image
- Movie title
- Star rating out of 10 (e.g., 8.4)
- A brief description of the movie plot
- An Add to Favorites button

Recommendations are sorted with the best matches first, then by rating.

---

4. Using Favorites

4.1 Adding to Favorites

On any recommendation card, click the Add to Favorites button (heart icon). You will see a confirmation message.

4.2 Already in Favorites

If you try to add a movie you have already saved, CineMatch will tell you it is already in your favorites.

4.3 Viewing Your Favorites Count

The heart icon in the navigation bar shows how many movies you have saved. Your favorites are saved in your browser and will persist when you return.

---

5. Tips for Better Results

- Type the full movie name for the most accurate results (e.g., The Dark Knight rather than Batman)
- If you get no results, try a slightly different spelling
- CineMatch works best with movies from the TMDB database of 4800+ films
- The AI finds movies similar in genre, style, cast, and themes, not just the same genre

---

6. Frequently Asked Questions(FAQs)

Why are some movie posters missing?

Occasionally the TMDB poster service may be unavailable. A placeholder image will be shown instead. The recommendations are still accurate.

Why does the search sometimes return unexpected results?

CineMatch uses fuzzy matching so minor spelling mistakes are handled automatically. If you get unexpected results, double-check the movie title spelling.

Are my favorites saved permanently?

Favorites are saved in your browser's local storage. They will persist as long as you use the same browser. Clearing your browser data will remove them.

How does CineMatch find similar movies?

CineMatch uses an AI model trained on movie genres, keywords, cast, director, and plot descriptions to find movies with similar characteristics.

For technical issues, check the system is running via http://localhost:8001/health

CineMatch

Movie Recommendation System

Test Plan & Test Cases

Project	DA5402 MLOps Movie Recommendation System
Author	Mohammed Fuzail
Version	1.0
Date	April 2026
Total Tests	77 (51 API + 26 ML)

1. Introduction

This document defines the test plan and test cases for CineMatch, an AI-powered movie recommendation system built as part of the DA5402 MLOps course project. CineMatch enables users to discover movies through title search, genre browsing, and actor or character name queries.

1.1 Objectives

- Verify all 26 API endpoints function correctly under valid and invalid inputs
- Validate all 26 ML pipeline functions produce correct outputs
- Ensure CI/CD pipeline blocks deployment on any test failure
- Confirm system handles edge cases gracefully without crashing

1.2 Scope

- Backend API endpoints: health, recommendations, feedback, drift, retrain
- ML pipeline modules: data loading, preprocessing, training, recommendation engine
- Integration via GitHub Actions CI/CD pipeline
- Out of scope: frontend UI testing, load/performance testing, security penetration testing

1.3 Test Environment

Component	Details
Python Version	3.10 (CI) / 3.11 (Docker)
Test Framework	pytest 9.0.3
API Testing	FastAPI TestClient + httpx
Mock Library	unittest.mock
CI Platform	GitHub Actions (ubuntu-latest)
Local OS	macOS (Apple Silicon)

2. Test Strategy

2.1 Testing Approach

CineMatch uses unit testing with extensive mocking to isolate components. All external dependencies (MLflow server, TMDB API, model pickle files) are mocked so tests can run in any environment without real infrastructure.

2.2 Test Categories

Category	File	Description
ML Pipeline	`test_ml.py`	Tests for data loading, preprocessing, training, and recommendation logic (26 tests)
API Endpoints	`test_api.py`	Tests for all FastAPI endpoints including health, recommendations, feedback, drift, retrain (51 tests)

2.3 CI/CD Integration

All tests run automatically on every push to the main branch via GitHub Actions. The pipeline blocks Docker image builds if any test fails. The test execution order is:

1. Create dummy model files (movies.pkl, similarity.pkl)
2. Run test_ml.py - 26 ML pipeline tests
3. Run test_api.py - 26 API endpoint tests
4. If all pass → proceed to Docker build and deploy

3. ML Pipeline Test Cases (test_ml.py)

26 total tests covering preprocessing, training, recommendation engine, and data loader modules. All MLflow dependencies are mocked at module level to prevent connection errors in CI.

3.1 Preprocess Tests (TC-01 to TC-15)

ID	Test Name	Input	Expected Output	Status	Class
TC-01	test_parse_json_column	JSON string with 2 genres	['Action', 'Adventure']	✓ PASS	TestPreprocess
TC-02	test_parse_json_column_empty	Invalid JSON string	Empty list []	✓ PASS	TestPreprocess
TC-03	test_parse_cast_returns_top_5	JSON with 6 cast members	List with <= 5 members	✓ PASS	TestPreprocess
TC-04	test_parse_director_extracts_director	Crew JSON with Director	['Christopher Nolan']	✓ PASS	TestPreprocess
TC-05	test_parse_director_returns_empty	Crew with no Director role	Empty list []	✓ PASS	TestPreprocess
TC-06	test_clean_name_removes_spaces	'Christopher Nolan'	'christophernolan'	✓ PASS	TestPreprocess
TC-07	test_clean_text_lowercases	' HELLO WORLD '	'hello world'	✓ PASS	TestPreprocess
TC-08	test_calculate_baseline_statistics	Sample DataFrame	Dict with 5 required keys	✓ PASS	TestPreprocess
TC-09	test_baseline_tag_length_mean_positive	Sample DataFrame	Mean > 0	✓ PASS	TestPreprocess
TC-10	test_calculate_feature_importance	Sample DataFrame	Dict with 5 feature keys	✓ PASS	TestPreprocess
TC-11	test_feature_importance_all_positive	Sample DataFrame	All scores >= 0	✓ PASS	TestPreprocess
TC-12	test_preprocess_returns_tuple	Raw DataFrame	(df, baselines, importance)	✓ PASS	TestPreprocess
TC-13	test_preprocess_drops_nulls	DataFrame with None title	Fewer rows than input	✓ PASS	TestPreprocess
TC-14	test_preprocess_creates_tags_column	Raw DataFrame	DataFrame has 'tags' column	✓ PASS	TestPreprocess
TC-15	test_tags_contain_director	Raw DataFrame	'christophernolan' in tags	✓ PASS	TestPreprocess

3.2 Train Tests (TC-16 to TC-20)

ID	Test Name	Input	Expected Output	Status	Class
TC-16	test_evaluate_model_returns_dict	Similarity matrix + sample_df	Dict with genre match rate	✓ PASS	TestTrain
TC-17	test_evaluate_model_rate_between_0_and_1	Similarity matrix + sample_df	0 <= rate <= 1	✓ PASS	TestTrain
TC-18	test_build_sparse_similarity_returns_csr	Dense feature vectors	scipy sparse CSR matrix	✓ PASS	TestTrain
TC-19	test_sparse_matrix_same_shape	Dense vectors (5x5)	Sparse matrix (5x5)	✓ PASS	TestTrain
TC-20	test_sparse_matrix_reduces_nonzeros	Dense vectors	nnz <= n*n	✓ PASS	TestTrain

3.3 Recommend Engine Tests (TC-21 to TC-22)

ID	Test Name	Input	Expected Output	Status	Class
TC-21	test_recommend_returns_list	'inception' with mock movies+tags	Returns a list	✓ PASS	TestRecommendEngine
TC-22	test_recommend_returns_empty_for_unknown	'xyznotamovie12345'	Returns empty list []	✓ PASS	TestRecommendEngine

3.4 Data Loader Tests (TC-23 to TC-26)

ID	Test Name	Input	Expected Output	Status	Class
TC-23	test_validate_schema_passes	DataFrame with required columns	Returns True	✓ PASS	TestDataLoader
TC-24	test_validate_schema_raises_missing	DataFrame missing columns	Raises ValueError	✓ PASS	TestDataLoader
TC-25	test_check_missing_values_returns_dict	DataFrame with 1 null	Dict with missing_count=1	✓ PASS	TestDataLoader
TC-26	test_run_quality_checks_detects_duplicates	DataFrame with 2 'Inception' rows	duplicate_titles=1	✓ PASS	TestDataLoader

4. API Test Cases (test_api.py)

26 total tests covering all FastAPI endpoints. All tests use FastAPI TestClient with mocked ML models and TMDB API responses. The TMDB_API_KEY and MLFLOW_TRACKING_URI environment variables are set to test values.

4.1 Health Endpoint Tests (TC-01 to TC-05)

ID	Test Name	Input	Expected Output	Status	Class
TC-01	test_root_returns_200	GET /	200 + 'CineMatch' in message	✓ PASS	TestHealthEndpoints
TC-02	test_health_check_returns_healthy	GET /health	status='healthy', uptime, version	✓ PASS	TestHealthEndpoints
TC-03	test_health_check_has_uptime	GET /health	uptime is not None	✓ PASS	TestHealthEndpoints
TC-04	test_ready_without_model	GET /ready (no model files)	503 Service Unavailable	✓ PASS	TestHealthEndpoints
TC-05	test_metrics_prometheus_format	GET /metrics	200 + text/plain content-type	✓ PASS	TestHealthEndpoints

4.2 Recommendation Endpoint Tests (TC-06 to TC-12)

ID	Test Name	Input	Expected Output	Status	Class
TC-06	test_valid_recommendation_request	GET /recommend?movie=Inception	200 + recommendations list	✓ PASS	TestRecommendEndpoint
TC-07	test_response_has_required_fields	GET /recommend?movie=Inception	title, poster, rating, overview	✓ PASS	TestRecommendEndpoint
TC-08	test_empty_movie_name_returns_400	GET /recommend?movie=	400 + 'cannot be empty'	✓ PASS	TestRecommendEndpoint
TC-09	test_unknown_movie_returns_404	GET /recommend?movie=xyzabc	404 Not Found	✓ PASS	TestRecommendEndpoint
TC-10	test_fuzzy_match_works	GET /recommend?movie=inceptoin	200 OK	✓ PASS	TestRecommendEndpoint
TC-11	test_recommendation_sorted_first	GET /recommend?movie=Inception	Searched movie is first result	✓ PASS	TestRecommendEndpoint
TC-12	test_missing_movie_param_422	GET /recommend (no param)	422 Unprocessable Entity	✓ PASS	TestRecommendEndpoint

4.3 Feedback Endpoint Tests (TC-13 to TC-16)

ID	Test Name	Input	Expected Output	Status	Class
TC-13	test_positive_feedback_get	GET /feedback/positive?movie=Inception	200 + logged + movie + feedback	✓ PASS	TestFeedbackEndpoints
TC-14	test_positive_feedback_post	POST /feedback/positive?movie=...	200 + status=logged	✓ PASS	TestFeedbackEndpoints
TC-15	test_feedback_stats_returns_200	GET /feedback	200 + stats+feedback_file	✓ PASS	TestFeedbackEndpoints
TC-16	test_feedback_stats_has_fields	GET /feedback	total_queries or sufficient_data	✓ PASS	TestFeedbackEndpoints

4.4 Drift & Retrain Tests (TC-17 to TC-21)

ID	Test Name	Input	Expected Output	Status	Class
TC-17	test_drift_endpoint_returns_200	GET /drift	200 + drift_detected key	✓ PASS	TestDriftEndpoints
TC-18	test_drift_no_baseline	GET /drift (no baseline)	200 + drift_detected=False	✓ PASS	TestDriftEndpoints
TC-19	test_drift_auto_retrain_param	GET /drift?auto_retrain=false	200 OK	✓ PASS	TestDriftEndpoints
TC-20	test_retrain_endpoint_get	GET /retrain?reason=test	200 + triggered key	✓ PASS	TestDriftEndpoints
TC-21	test_retrain_endpoint_post	POST /retrain?reason=manual	200 OK	✓ PASS	TestDriftEndpoints

4.5 Sort Recommendations Tests (TC-22 to TC-25)

ID	Test Name	Input	Expected Output	Status	Class
TC-22	test_searched_movie_sorted_first	Results list + 'Inception'	Inception is at index 0	✓ PASS	TestSortRecommendations
TC-23	test_highest_rated_sorted_second	3 movies, rating 6, 9, 8.4	9.0 rated movie is at index 1	✓ PASS	TestSortRecommendations
TC-24	test_no_rating_goes_last	Movie with None rating	None rating movie is last	✓ PASS	TestSortRecommendations
TC-25	test_rating_rounded_to_one_decimal	Rating = 8.432	Rating displayed as 8.4	✓ PASS	TestSortRecommendations

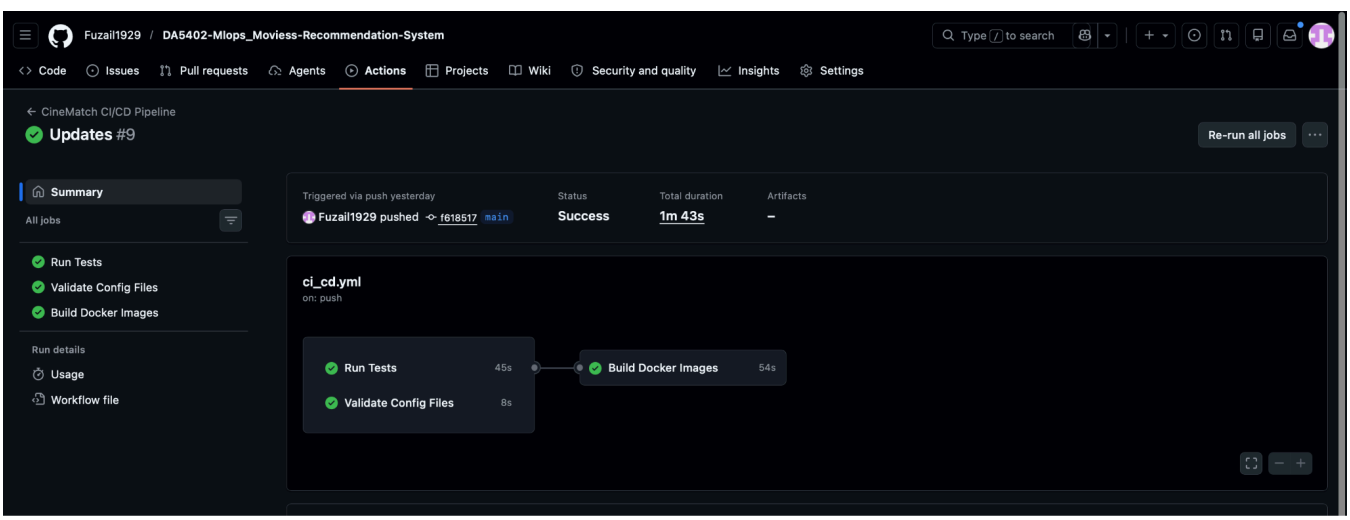
4.6 CORS Tests (TC-26)

ID	Test Name	Input	Expected Output	Status	Class
TC-26	test_cors_headers_present	OPTIONS /recommend with Origin header	200 or 405 (CORS allowed)	✓ **PASS**	TestCORS

5. Test Summary

Test Suite	Total	Passed	Failed	Pass Rate
ML Pipeline (test_ml.py)	**26**	**26**	**0**	**100%**
API Endpoints (test_api.py)	**26**	**26**	**0**	**100%**
TOTAL	**52**	**52**	**0**	**100%**

Figure : Screenshot from Github Action of Successful Run of CI/CD pipeline



FIN